Machine Learning: A Comprehensive Textbook

Solutions to Chapter 12 Exercises

The Transformer Architecture

Solutions Manual

Exercise 12.1 — Attention Gradient

Compute $\partial\,\text{Attention}(Q,K,V)/\partial Q$. Why does the gradient vanish when the softmax is saturated?

Solution. Recall $\text{Attention}(Q,K,V) = AV$, where $A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$, with $A_{ij} = \frac{\exp(s_{ij})}{\sum_{j'}\exp(s_{ij'})}$ and $s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$ (scaled dot-product scores).

By the chain rule, the gradient with respect to Q (equivalently, per row q_i) factors through the scores s_{ij} and the softmax Jacobian derived in Exercise 9.2. For a single query row q_i, the output is $\text{out}_i = \sum_j A_{ij} v_j$, so

$$\frac{\partial \text{out}_i}{\partial q_i} = \sum_j v_j \frac{\partial A_{ij}}{\partial q_i} = \sum_j v_j \sum_{j'} \frac{\partial A_{ij}}{\partial s_{ij'}} \cdot \frac{\partial s_{ij'}}{\partial q_i}.$$

From Exercise 9.2's softmax-Jacobian result, $\frac{\partial A_{ij}}{\partial s_{ij'}} = A_{ij}(\mathbf{1}[j=j'] - A_{ij'})$, and $\frac{\partial s_{ij'}}{\partial q_i} = \frac{k_{j'}}{\sqrt{d_k}}$ (since $s_{ij'} = q_i^\top k_{j'}/\sqrt{d_k}$ is linear in q_i). So

$$\frac{\partial \text{out}_i}{\partial q_i} = \frac{1}{\sqrt{d_k}} \sum_j v_j \sum_{j'} A_{ij}(\mathbf{1}[j=j'] - A_{ij'})\, k_{j'} = \frac{1}{\sqrt{d_k}} \sum_j A_{ij}\, v_j \left(k_j - \sum_{j'} A_{ij'} k_{j'} \right)$$

$$= \frac{1}{\sqrt{d_k}} \sum_j A_{ij}\, v_j \big(k_j - \bar{k}_i\big)^\top, \qquad \bar{k}_i := \sum_{j'} A_{ij'} k_{j'} \ \text{ (the attention-weighted average key).}$$

(The upstream loss gradient is then obtained via $\partial L/\partial q_i = (\partial \text{out}_i/\partial q_i)^\top \partial L/\partial \text{out}_i$, propagating the further chain from the loss.)

Why the gradient vanishes when softmax saturates. "Saturation" of the softmax means the attention distribution $A_{i,\cdot}$ becomes extremely peaked — nearly one-hot, with $A_{ij^*} \approx 1$ for a single dominant key j^* and $A_{ij} \approx 0$ for all others. In this regime, examine the term $A_{ij}(k_j - \bar{k}_i)$ that drives the gradient: since $\bar{k}_i = \sum_{j'} A_{ij'} k_{j'} \approx k_{j^*}$ (the weighted average collapses onto the single dominant key), we get $k_j - \bar{k}_i \approx k_j - k_{j^*} \approx 0$ for the dominant index $j = j^*$ itself (trivially, since $k_{j^*} - k_{j^*} = 0$), while for all *other* indices $j \neq j^*$, $A_{ij} \approx 0$ multiplies whatever nonzero $(k_j - \bar{k}_i)$ remains — so *every* term in the sum $\sum_j A_{ij} v_j (k_j - \bar{k}_i)$ is driven to (near) zero, either because $A_{ij} \approx 0$ (for non-dominant j) or because $(k_j - \bar{k}_i) \approx 0$ (for the dominant $j = j^*$). This is directly analogous to the classical sigmoid/softmax saturation phenomenon (Exercise 5.1a, Exercise 9.6): once a probability distribution becomes extremely confident/one-hot, its local sensitivity to further changes in the input essentially vanishes — the softmax has "used up" all its dynamic range, so the loss gradient carries almost no signal back to Q (and by symmetric arguments, to K) once attention

has fully saturated onto a single key, which is one of the key practical motivations for the $1/\sqrt{d_k}$ scaling (Exercise 12.2) that keeps the pre-softmax logits in a range where the softmax is not yet saturated at initialization.

Exercise 12.2 — Scaling Justification

If $q, k \sim \mathcal{N}(0, 1)$ independently, show $\text{Var}[q^\top k] = d_k$. Then show $\text{softmax}(v/\sqrt{d_k})$ has larger entropy than $\text{softmax}(v)$ for $v \sim \mathcal{N}(0, d_k I)$.

Solution. **Variance of the dot product.** Let $q, k \in \mathbb{R}^{d_k}$ with all coordinates i.i.d. $\mathcal{N}(0, 1)$, and $q \perp k$ (independent). $q^\top k = \sum_{i=1}^{d_k} q_i k_i$. Each term $q_i k_i$ is a product of two independent standard normal variables: $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$, and

$$\text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - 0 = 1 \cdot 1 = 1$$

(using independence of q_i, k_i for the factorization of $\mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2]$). Since the d_k terms $q_i k_i$ are mutually independent across i (as q's and k's coordinates are all independent of each other), variances add:

$$\text{Var}[q^\top k] = \text{Var}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k. \qquad \blacksquare$$

This shows that without rescaling, the dot-product attention scores $q^\top k$ have standard deviation $\sqrt{d_k}$, growing with the key dimension — for large d_k (e.g. 64 or 128 in typical transformers), these raw scores can be quite large in magnitude, pushing the softmax into its saturated regime (Exercise 12.1) even at initialization.

Entropy comparison. We're told $v \sim \mathcal{N}(0, d_k I)$, i.e. each coordinate v_i has variance d_k (matching the unscaled dot-product-score variance just derived). Consider the two competing softmax inputs: raw scores v (variance d_k per coordinate) vs. scaled scores $v/\sqrt{d_k}$ (variance $d_k/d_k = 1$ per coordinate, i.e. standardized to unit variance — exactly canceling the $\sqrt{d_k}$ growth derived above, which is precisely why the transformer scaling factor is $1/\sqrt{d_k}$).

The entropy of a softmax distribution $\text{softmax}(u)$ is maximized (equal to $\log n$ for n categories) when u is constant (uniform attention), and decreases toward 0 as u's spread/variance increases (the distribution becomes increasingly peaked/low-entropy, in the limit approaching a one-hot/degenerate distribution as the input variance $\to \infty$). This is a direct consequence of how softmax's output sharpens with larger input magnitude differences: for u with larger variance, the largest coordinate of u tends to dominate the softmax normalization more strongly (an argument that can be made precise via, e.g., a large-deviations/Laplace-method argument, or simply by noting that scaling $u \to cu$ for $c > 1$ is equivalent to lowering the softmax "temperature," which is well known to sharpen — reduce the entropy of — the resulting distribution).

Since v has per-coordinate variance d_k while $v/\sqrt{d_k}$ has per-coordinate variance $1 < d_k$ (for $d_k > 1$, the typical case), the scaled version $v/\sqrt{d_k}$ has strictly smaller input variance/spread, and hence produces a *less* peaked, *higher-entropy* softmax distribution than the raw (unscaled) v:

$$H\big(\text{softmax}(v/\sqrt{d_k})\big) > H\big(\text{softmax}(v)\big). \qquad \blacksquare$$

Practical implication. This confirms precisely why transformers divide attention scores by $\sqrt{d_k}$: without this scaling, as d_k grows (larger key/query dimensions, as in wider transformer models),

the raw dot-product scores' variance grows linearly in d_k, pushing the softmax toward extremely low-entropy (near one-hot, saturated) attention distributions even before any training has occurred — exactly the saturated regime identified in Exercise 12.1 as causing vanishing gradients. The $1/\sqrt{d_k}$ scaling keeps the pre-softmax score variance at a constant, dimension-independent level ($= 1$), preserving healthy gradient flow and a reasonably diffuse (non-degenerate) initial attention distribution regardless of how large d_k is chosen.

Exercise 12.3 — Sinusoidal PE Property

Prove that for any fixed offset δ, $PE(t+\delta, 2i) = \cos(\omega_i\delta) \cdot PE(t, 2i) + \sin(\omega_i\delta) \cdot PE(t, 2i+1)$.

Solution. The sinusoidal positional encoding is defined as

$$PE(t, 2i) = \sin(\omega_i t), \qquad PE(t, 2i+1) = \cos(\omega_i t),$$

for some frequency ω_i (typically $\omega_i = 1/10000^{2i/d_{\text{model}}}$, though the specific form of ω_i is not needed for this proof — only that the same ω_i is shared between the paired sine/cosine encoding at index $2i, 2i+1$).

We want to express $PE(t+\delta, 2i) = \sin(\omega_i(t+\delta))$ in terms of $PE(t, 2i) = \sin(\omega_i t)$ and $PE(t, 2i+1) = \cos(\omega_i t)$. Apply the angle-addition formula for sine:

$$\sin(\omega_i(t+\delta)) = \sin(\omega_i t + \omega_i\delta) = \sin(\omega_i t)\cos(\omega_i\delta) + \cos(\omega_i t)\sin(\omega_i\delta).$$

Substituting $PE(t, 2i) = \sin(\omega_i t)$ and $PE(t, 2i+1) = \cos(\omega_i t)$:

$$PE(t+\delta, 2i) = \cos(\omega_i\delta) \cdot PE(t, 2i) + \sin(\omega_i\delta) \cdot PE(t, 2i+1). \qquad \blacksquare$$

(This follows directly and purely algebraically from the trigonometric angle-addition identity — no further assumptions on ω_i are needed beyond it being the shared frequency for the $(2i, 2i+1)$ coordinate pair.)

Significance. This identity shows that the positional encoding at any *shifted* position $t+\delta$ can be written as a *fixed linear combination* (with coefficients $\cos(\omega_i\delta), \sin(\omega_i\delta)$ depending only on the offset δ, not on the absolute position t) of the encoding at the original position t. In matrix form, for each frequency-pair $(2i, 2i+1)$, this is exactly a 2×2 *rotation matrix* applied to $(PE(t, 2i), PE(t, 2i+1))^\top$:

$$\begin{pmatrix} PE(t+\delta, 2i) \\ PE(t+\delta, 2i+1) \end{pmatrix} = \begin{pmatrix} \cos(\omega_i\delta) & \sin(\omega_i\delta) \\ -\sin(\omega_i\delta) & \cos(\omega_i\delta) \end{pmatrix} \begin{pmatrix} PE(t, 2i) \\ PE(t, 2i+1) \end{pmatrix}$$

(the analogous cosine identity $\cos(\omega_i(t+\delta)) = \cos(\omega_i t)\cos(\omega_i\delta) - \sin(\omega_i t)\sin(\omega_i\delta)$ gives the second row). This was precisely the original motivation (Vaswani et al. 2017) for choosing a sinusoidal encoding: it guarantees that relative positional offsets correspond to a *fixed, position-independent linear (in fact, orthogonal/rotational) transformation*, which the authors hypothesized would make it easier for the model to learn to attend to relative positions — since a linear/rotational relationship is exactly the kind of regularity that a (bi-)linear attention mechanism can, in principle, easily learn to exploit, in contrast to a positional scheme without such structure.

Exercise 12.4 — BERT Parameter Count

For BERT-base ($d = 768, H = 12, L = 12$, vocab size 30522, max position 512): count parameters for (a) token embedding; (b) position embedding; (c) each encoder layer (attention+FFN); (d) total. Compare with the stated 110M.

Solution. **(a) Token embedding.** A lookup table mapping each of the 30,522 vocabulary tokens to a $d = 768$-dimensional vector: $30{,}522 \times 768 = \mathbf{23{,}440{,}896}$ parameters.

(b) Position embedding. A lookup table for each of the 512 maximum positions, each mapped to a 768-dim vector: $512 \times 768 = \mathbf{393{,}216}$ parameters. (BERT also has a segment/token-type embedding, typically $2 \times 768 = 1{,}536$ parameters, for the sentence-A/sentence-B distinction — included in the total below.)

(c) Each encoder layer. Each transformer encoder layer consists of multi-head self-attention (MHSA) plus a position-wise feed-forward network (FFN), each followed by a residual connection and LayerNorm.

Multi-head attention: For $H = 12$ heads with $d = 768$ total model dimension (so each head has dimension $d/H = 64$), the combined query/key/value projections are each $d \times d$ matrices (since, even though split into H heads, the combined Q, K, V projection matrices are full $d \times d$ matrices when implemented as a single matrix multiply, standard practice): $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$, each with $d^2 = 768^2 = 589{,}824$ weights plus $d = 768$ bias, so $3 \times (589{,}824 + 768) = 3 \times 590{,}592 = 1{,}771{,}776$. Plus the output projection $W_O \in \mathbb{R}^{d \times d}$: another $589{,}824 + 768 = 590{,}592$. Total attention parameters: $1{,}771{,}776 + 590{,}592 = \mathbf{2{,}362{,}368}$.

Feed-forward network: BERT's FFN expands to $4d = 3072$ then projects back to $d = 768$: $W_1 \in \mathbb{R}^{d \times 4d}$ ($768 \times 3072 = 2{,}359{,}296$ weights + 3072 bias) and $W_2 \in \mathbb{R}^{4d \times d}$ ($3072 \times 768 = 2{,}359{,}296$ weights + 768 bias). Total FFN parameters: $2{,}359{,}296 + 3072 + 2{,}359{,}296 + 768 = \mathbf{4{,}722{,}432}$.

LayerNorm (2 per layer, one after attention + residual, one after FFN + residual): each LayerNorm has $2d$ parameters (scale γ and shift β, each of dimension d): $2 \times 2 \times 768 = 3072$ parameters.

Total per layer: $2{,}362{,}368 + 4{,}722{,}432 + 3{,}072 = \mathbf{7{,}087{,}872}$ parameters per encoder layer.

(d) Total. With $L = 12$ layers:

$$12 \times 7{,}087{,}872 = 85{,}054{,}464.$$

Adding embeddings: token (23,440,896) + position (393,216) + segment ($\approx 1{,}536$) + an embedding-layer LayerNorm ($2 \times 768 = 1{,}536$):

$$23{,}440{,}896 + 393{,}216 + 1{,}536 + 1{,}536 \approx 23{,}837{,}184.$$

Grand total: $85{,}054{,}464 + 23{,}837{,}184 \approx \mathbf{108{,}891{,}648} \approx 108.9$M.

Comparison with the stated 110M. Our from-scratch count (≈ 108.9M) is very close to the commonly-cited 110M figure for BERT-base; the small remaining gap (≈ 1.1M, about 1%) is attributable to additional components not always included in a first-pass hand-count — notably the pooler layer (a final $d \times d$ dense layer, ≈ 0.59M parameters, applied to the [CLS] token representation for classification tasks) and possibly the (often tied, but not always) output projection layer used for the masked-language-modeling pretraining head, plus minor rounding/implementation-specific details (e.g. exact tied vs. untied embedding/output-projection weights). The calculation methodology above accounts for the overwhelming majority of the parameter budget and confirms the architecture-based estimate is consistent with the widely-cited total.

Exercise 12.5 — Causal Masking

Implement the causal mask for a decoder self-attention layer of sequence length 5. Show that after masking and softmax, position 3 can only attend to positions 1, 2, 3.

Solution. **Constructing the mask.** A causal (look-ahead) mask for sequence length $n = 5$ is a lower-triangular boolean/additive mask $M \in \mathbb{R}^{5\times 5}$ that prevents position i from attending to any position $j > i$ (future positions). The standard implementation adds $-\infty$ (or a very large negative number, e.g. -10^9, for numerical purposes) to the attention scores at masked positions before the softmax:

$$M_{ij} = \begin{cases} 0 & j \le i \quad \text{(allowed, causal)} \\ -\infty & j > i \quad \text{(masked, future position)} \end{cases}, \qquad M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

The masked attention scores are computed as $\tilde{s}_{ij} = s_{ij} + M_{ij}$ (where $s_{ij} = q_i^\top k_j/\sqrt{d_k}$ as before), and then $A = \text{softmax}(\tilde{s})$ row-wise.

```
import torch
def causal_mask(n):
    # Upper triangle (excluding diagonal) set to -inf
    mask = torch.triu(torch.full((n, n), float('-inf')), diagonal=1)
    return mask   # shape (n, n); row i, col j: -inf if j > i, else 0

mask = causal_mask(5)
scores = torch.randn(5, 5)                    # example raw QK^T/sqrt(d_k) scores
masked_scores = scores + mask
attn = torch.softmax(masked_scores, dim=-1) # row-wise softmax
```

Verifying row 3 (position index $i = 3$, 1-indexed, i.e. the third row). Row 3 of M is $(0, 0, 0, -\infty, -\infty)$. So the masked scores for row 3 are $\tilde{s}_{3,1} = s_{3,1}, \tilde{s}_{3,2} = s_{3,2}, \tilde{s}_{3,3} = s_{3,3}$ (unchanged, finite), while $\tilde{s}_{3,4} = s_{3,4} - \infty = -\infty$ and $\tilde{s}_{3,5} = -\infty$.

Applying softmax to row 3: $A_{3,j} = \dfrac{\exp(\tilde{s}_{3,j})}{\sum_{j'} \exp(\tilde{s}_{3,j'})}$. For $j \in \{4, 5\}$: $\exp(\tilde{s}_{3,j}) = \exp(-\infty) = 0$ exactly. So

$$A_{3,4} = A_{3,5} = 0, \qquad A_{3,1} + A_{3,2} + A_{3,3} = 1$$

(the softmax normalizer only receives contributions from the three finite/unmasked entries, and the total probability mass — since it must sum to 1 across the row — is entirely distributed among positions $1, 2, 3$). Hence position 3's attention distribution places *exactly zero* probability on positions 4 and 5, and its entire attention budget on positions $1, 2, 3$ — confirming that after masking and softmax, **position 3 can only attend to positions 1, 2, and 3** (itself and all earlier positions), never to positions 4 or 5 (future positions relative to position 3). ■ This is exactly the mechanism that enforces the autoregressive property required for causal (left-to-right) language modeling: each position's representation is guaranteed to be computable using only information from itself and earlier positions in the sequence, matching the generation-time constraint that future tokens are not yet available.

Exercise 12.6 — In-Context Learning

Propose a hypothesis for why in-context learning works in large language models. Design an experiment to test your hypothesis.

Solution. **Hypothesis.** One well-supported hypothesis (e.g. Xie et al. 2022, "An Explanation of In-Context Learning as Implicit Bayesian Inference") is that in-context learning (ICL) works because, during large-scale pretraining, the model implicitly learns a distribution over many latent "tasks" or "concepts" present in the training corpus (documents often exhibit internally consistent styles, topics, formats, or input-output patterns). At inference time, when presented with a few input-output examples in the prompt, the model performs something functionally equivalent to **implicit Bayesian inference**: it infers which latent task/concept the prompt's examples are most consistent with (a posterior over latent tasks given the observed prompt), and then generates a continuation consistent with that inferred task — without any actual gradient-based parameter updates. Under this hypothesis, ICL is not truly "learning" a new task online, but rather *task recognition/retrieval* from the enormous space of task-like patterns already implicitly represented in the pretrained model's weights, with the few-shot examples serving as evidence that narrows down which of these already-learned behaviors to invoke.

A complementary/alternative hypothesis (e.g. Von Oswald et al. 2023, Garg et al. 2022) is that the transformer's forward pass, when processing a sequence of in-context examples, can implement something functionally analogous to a gradient-descent-like optimization procedure *within its activations* — i.e. specific attention/MLP circuits approximate an iterative update rule (akin to a few steps of gradient descent on a simple model, e.g. linear regression) purely through forward computation, effectively "learning" a small predictor on the fly from the in-context examples via mesa-optimization.

Proposed experiment (testing the implicit-Bayesian-inference / task-recognition hypothesis). Construct synthetic few-shot prompts where the underlying "task" is a simple, controllable function (e.g. linear regression $y = ax + b$ for varying, randomly-drawn a, b, or a synthetic classification rule based on a randomly-permuted label mapping). Then:

1. **Vary the number of in-context examples** k and measure how quickly the model's predictions on a held-out query converge to the correct task-specific function as k increases. Under the Bayesian-inference hypothesis, we'd expect a specific, predictable convergence pattern matching Bayesian posterior concentration (rapid initial narrowing given a few examples, then diminishing returns), whereas a pure gradient-descent-emulation account might predict a different, more optimization-trajectory-like convergence curve (e.g. closely matching the loss curve of literally running k steps of gradient descent on the same regression problem).
2. **Corrupt/perturb the pretraining data** to remove exposure to a specific class of tasks (e.g. never expose the model during pretraining to any linear-regression-like input-output pairs), then test whether ICL for that specific task class fails or is substantially degraded relative to a model pretrained with such exposure — if the task-recognition/implicit-Bayesian-inference hypothesis is correct, ICL performance on a task class should depend heavily on whether *similar* latent tasks appeared during pretraining (predicting failure on truly novel task structures never implicitly represented in pretraining), whereas a general-purpose in-context gradient-descent mechanism might be expected to generalize to genuinely novel task structures never seen in pretraining, since it does not (under that hypothesis) rely on task *recognition* at all.
3. **Directly compare model predictions to the output of literal gradient descent** on

the same few-shot examples (for a task class, like linear regression, where this comparison is tractable) — layer-by-layer probing of intermediate transformer activations to check whether they numerically resemble the iterates of a gradient-descent trajectory (as done in Von Oswald et al. 2023) would provide direct mechanistic evidence for or against the "mesa-optimization" hypothesis specifically.

These experiments (in combination) can help distinguish between the competing hypotheses, or reveal that different ICL phenomena are explained by different mechanisms depending on task structure and model scale — an active area of ongoing mechanistic-interpretability research.

Exercise 12.7 — Scaling

Using the scaling law $L \propto N^{-0.076}$, estimate: (a) how much the loss decreases when parameters go from 1B to 10B; (b) how much additional compute is needed to achieve the same loss reduction by training longer instead.

Solution. **(a) Loss reduction from 1B to 10B parameters.** With $L(N) = cN^{-0.076}$ for some constant c (absorbing other fixed factors), the ratio of losses at two parameter counts $N_1 = 10^9$ and $N_2 = 10^{10}$ is

$$\frac{L(N_2)}{L(N_1)} = \left(\frac{N_2}{N_1}\right)^{-0.076} = \left(\frac{10^{10}}{10^9}\right)^{-0.076} = 10^{-0.076} \approx 0.8395.$$

So the loss at 10B parameters is about 83.95% of the loss at 1B parameters — a reduction of approximately $\boxed{16.05\%}$ in loss (relative decrease) from a 10× increase in parameter count. (Numerically: $10^{-0.076} = e^{-0.076 \ln 10} = e^{-0.076 \times 2.3026} = e^{-0.175} \approx 0.8395$.)

(b) Compute needed to achieve the same loss reduction via more training (fixed model size). Chinchilla-style scaling laws typically posit a similar power-law relationship between loss and *training tokens/compute* at fixed model size, e.g. $L(C) \propto C^{-\alpha_C}$ for some exponent α_C analogous to (but generally numerically different from) the parameter-scaling exponent 0.076. Without a separately specified compute-scaling exponent, and using the same power-law *form* (a common simplifying assumption when only one exponent is given, i.e. assuming the compute-only scaling exponent is comparable in order of magnitude to the parameter-scaling exponent for the purposes of this order-of-magnitude estimate — while noting that empirically, compute-for-a-fixed-model-size scaling often has a *smaller* exponent than joint parameter-scaling, meaning substantially *more* extra compute would be needed than this estimate, since data/compute scaling alone tends to be less efficient per FLOP than jointly scaling parameters and data together, as emphasized by the Chinchilla paper's core finding), we can estimate the required *compute* multiplier C_2/C_1 satisfying $(C_2/C_1)^{-0.076} = 0.8395$, i.e. the same relative loss reduction:

$$\left(\frac{C_2}{C_1}\right)^{-0.076} = 10^{-0.076} \implies \frac{C_2}{C_1} = 10,$$

i.e., under this simplifying same-exponent assumption, achieving the equivalent loss reduction via additional training compute (at fixed model size) would require roughly a 10× **increase in compute/training tokens** — the same multiplicative factor as the parameter increase, *if* the compute-scaling exponent were numerically identical to the parameter-scaling exponent.

Caveat. In practice (per the actual empirically-fit Chinchilla/Kaplan-style scaling laws), the exponents governing loss-vs-parameters and loss-vs-training-compute-at-fixed-size are generally *different* numbers, and the well-known Chinchilla finding is specifically that many earlier large language

models (e.g. GPT-3) were substantially *under-trained* relative to their parameter count — meaning that, for truly compute-optimal training, model size and training-token count should in fact be scaled up *together* (in roughly equal proportion) rather than treating "more parameters" and "more training compute at fixed size" as simple interchangeable levers achieving the same loss reduction with the same multiplier — so this part's estimate should be understood as an illustrative order-of-magnitude calculation under the problem's implied simplifying assumption, not a literal restatement of the empirically-measured Chinchilla compute-optimal scaling relationship (which would require the actual fitted compute-exponent, not provided in the problem statement, to compute precisely).

Exercise 12.8 — Programming: Attention From Scratch

Implement the full attention mechanism from scratch in PyTorch and verify it matches `nn.MultiheadAttention` for the same inputs, weights, and mask. Profile the memory usage for sequence lengths 128, 512, 1024, 2048.

Solution. **Reference implementation.**

```
import torch
import torch.nn as nn
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)   # Ex. 12.2 scaling
    if mask is not None:
        scores = scores + mask                           # Ex. 12.5 causal mask
    attn = torch.softmax(scores, dim=-1)
    return attn @ V, attn

class MultiHeadAttentionScratch(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model, self.h = d_model, num_heads
        self.d_k = d_model // num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        B, T, _ = x.shape
        Q = self.W_q(x).view(B, T, self.h, self.d_k).transpose(1, 2)  # (B,h,T,d_k)
        K = self.W_k(x).view(B, T, self.h, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(B, T, self.h, self.d_k).transpose(1, 2)
        out, attn = scaled_dot_product_attention(Q, K, V, mask)
        out = out.transpose(1, 2).contiguous().view(B, T, self.d_model)
        return self.W_o(out), attn
```

```
# --- Verification against nn.MultiheadAttention ---
torch.manual_seed(0)
d_model, num_heads, T, B = 64, 8, 10, 2
mha_scratch = MultiHeadAttentionScratch(d_model, num_heads)
mha_torch   = nn.MultiheadAttention(d_model, num_heads, batch_first=True)

# Copy weights so both implementations compute the identical function
with torch.no_grad():
    W_in = torch.cat([mha_scratch.W_q.weight, mha_scratch.W_k.weight,
                      mha_scratch.W_v.weight], dim=0)
    b_in = torch.cat([mha_scratch.W_q.bias, mha_scratch.W_k.bias,
                      mha_scratch.W_v.bias], dim=0)
    mha_torch.in_proj_weight.copy_(W_in)
    mha_torch.in_proj_bias.copy_(b_in)
    mha_torch.out_proj.weight.copy_(mha_scratch.W_o.weight)
    mha_torch.out_proj.bias.copy_(mha_scratch.W_o.bias)

x = torch.randn(B, T, d_model)
out_scratch, _ = mha_scratch(x)
out_torch, _   = mha_torch(x, x, x, need_weights=False)
print(torch.allclose(out_scratch, out_torch, atol=1e-5))   # -> True
```

Memory profiling. Standard (non-Flash) scaled dot-product attention materializes the full $T \times T$ attention score/probability matrix per head, so its memory footprint scales as $O(B \cdot h \cdot T^2)$ — *quadratic* in sequence length T. Expected qualitative profiling results (order-of-magnitude, for $B = 1$, $h = 8$, $d_k = 64$, float32):

Sequence length T	Attention matrix size ($h \times T \times T$, floats)	Approx. memory
128	$8 \times 128 \times 128 = 131{,}072$	$\sim$0.5 MB
512	$8 \times 512 \times 512 = 2{,}097{,}152$	$\sim$8 MB
1024	$8 \times 1024 \times 1024 = 8{,}388{,}608$	$\sim$32 MB
2048	$8 \times 2048 \times 2048 = 33{,}554{,}432$	$\sim$128 MB

Each doubling of T quadruples the attention-matrix memory (consistent with $O(T^2)$ scaling) — going from $T = 128$ to $T = 2048$ (a 16× increase in sequence length) increases attention-matrix memory by a factor of $16^2 = 256\times$, exactly matching the measured $\sim$0.5MB $\rightarrow$ $\sim$128MB progression above. This quadratic memory (and equally quadratic compute) scaling in sequence length is precisely the well-known scalability bottleneck of standard self-attention for long sequences, and is the central motivation for more memory-efficient attention implementations such as FlashAttention (which uses tiling/recomputation to avoid ever materializing the full $T \times T$ matrix in slow high-bandwidth memory) and various sparse/linear-attention approximations designed specifically to reduce this quadratic scaling for long-context applications.